

Diagramas de Árbol — Trazas Recursivas

Universidad de Caldas · Técnicas de Programación
Patrones: Índice · Divide & Conquer · Incluir/Excluir

Cómo dibujar el árbol de llamadas (pasos)

- 1) Define el **estado** que cambia: índice i , rango $l..r$, o conjunto actual.
- 2) Dibuja la **raíz** con la llamada inicial y sus parámetros.
- 3) Crea un nodo por **subllamada**. Repite hasta llegar al **caso base**.
- 4) Marca el caso base con ✓ y anota **qué retorna**.
- 5) Regresa combinando las respuestas de los hijos y escribe el **retorno** en cada nodo.
-
- Consejo: usa entradas mínimas para que el árbol quepa en una hoja.

Ejemplo (ÍNDICE): suma([2,4,1])

- Función:
- `def suma(a, i=0):`
- `if i == len(a): return 0`
- `return a[i] + suma(a, i+1)`
-
- Árbol de llamadas y retornos:
- `suma([2,4,1], i=0)`
- `└ suma(i=1)`
- `└ suma(i=2)`
- `└ suma(i=3) ✓ retorna 0`
- `retorna 1 + 0 = 1`
- `retorna 4 + 1 = 5`
- `retorna 2 + 5 = 7`

**Ejemplo (Divide
& Conquer):
búsqueda
binaria en
[1,3,5,7]
buscando 5**

- Función (esquema): `bbin(a, x, l, r)`
- Caso base: $l > r \rightarrow -1$; $a[m] == x \rightarrow m$
-
- Árbol (parámetros abreviados):
- `bbin(l=0, r=3)` $m=1$, $a[m]=3 < 5 \rightarrow$ derecha
- $\hookleftarrow$ `bbin(l=2, r=3)` $m=2$, $a[m]=5 == 5 \checkmark$ retorna 2
- retorna 2

Ejemplo (Incluir/Excluir): $\text{power_set}([a, b])$

- $\text{go}(i=0, \text{actual}=[])$
- $\vdash (\text{excluir } a) \text{ go}(i=1, \text{actual}=[])$
- $\mid \vdash (\text{excluir } b) \text{ go}(i=2, []) \checkmark \text{ registra } []$
- $\mid \vdash (\text{incluir } b) \text{ go}(i=2, [b]) \checkmark \text{ registra } [b]$
- $\vdash (\text{incluir } a) \text{ go}(i=1, [a])$
- $\vdash (\text{excluir } b) \text{ go}(i=2, [a]) \checkmark \text{ registra } [a]$
- $\vdash (\text{incluir } b) \text{ go}(i=2, [a, b]) \checkmark \text{ registra } [a, b]$

Plantilla en blanco — Patrón ÍNDICE

▪ Problema:

▪ Estado: $i = ______$ Entrada:

▪

▪ Raíz:

▪ `solve(i=___)`

▪ $\sqsubset$ `solve(i=___)`

▪ $\sqsubset$ `solve(i=___)`

▪ $\sqsubset$ `solve(i=___)` ✓ retorna _____

▪ retorna: _____

▪ retorna: _____

▪ retorna: _____

▪

Plantilla en blanco — Divide & Conquer

- Problema: _____
- Estado: $l=$ __, $r=$ __ (o cortes por mitades)
-
- Raíz:
- $bb(l=$ __, $r=$ __)
- $\vdash$ subproblema A: _____
- $\mid \quad \sqsubset \dots$
- $\sqsubset$ subproblema B: _____
- $\quad \sqsubset \dots$
- Combinar(A, B) $\rightarrow$ retorna: _____

Plantilla en
blanco —
Incluir/Excluir

- ```
Estado: i=__, actual=_____

Raíz:

go(i=__, actual=[])
├ excluir elemento[i]
│ └ go(i=__, actual=[])
└ incluir elemento[i]
 └ go(i=__, actual=[elemento[i]])

Caso base i==n → registrar/retornar:
```

# Snippet: decorador para trazar árboles (texto)

```
def trace(fn):
 from functools import wraps
 def indent(depth): return '|' * (depth-1) + ('| '
 if depth>0 else '')
 @wraps(fn)
 def wrapper(*args, **kwargs):
 depth = wrapper.depth
 print(f"{indent(depth)}{fn.__name__}{args}
 {kwargs}")
 wrapper.depth += 1
 res = fn(*args, **kwargs)
 wrapper.depth -= 1
 print(f"{indent(depth)}→ {res}")
 return res
 wrapper.depth = 0
 return wrapper

Uso:
@trace
def suma(a, i=0):
 if i==len(a): return 0
 return a[i] + suma(a, i+1)

suma([2,4,1]) # imprime el árbol y los retornos
```